

Oracle -> BigQuery dbt Migration

Agent Knowledge Base

Medallion Architecture . CrewAI Pipeline . PL/SQL Conversion Reference

BRONZE

SILVER

GOLD

Purpose	Reference knowledge for CrewAI agents executing Oracle-to-BigQuery migration
Architecture	Medallion (Bronze / Silver / Gold)
Target Platform	Google BigQuery + dbt Core
Source Platform	Oracle Database (PL/SQL)
Agent Framework	CrewAI multi-agent pipeline
Key Insight	Most Oracle procedures are orchestration overhead. dbt run replaces them. Only MERGE_UPSE

1. Pipeline Overview

The migration is executed as a sequential six-phase CrewAI pipeline. Each phase is owned by a dedicated specialist agent. The Supervisor Agent orchestrates all phases, enforces quality gates, and is the sole agent with `allow_delegation: true`.

Phase	Agent	Task	Key Output
0 - Pre-flight	supervisor_agent	supervise_migration_task	Connectivity verified, config loaded
1 - Discovery	discovery_agent	discover_oracle_schema_task	Full inventory + medallion tags + proc_patterns
2 - Classification	classifier_agent	classify_complexity_task	migration_strategy per proc + complexity tier
3 - Schema Conversion	schema_conversion_agent	convert_schemas_task	BQ DDL (Bronze only) + dbt model stubs (a
4 - Code Generation	codegen_agent	convert_procs_task	Incremental models + snapshots + macros
5 - Validation	validator_agent	validate_migrations_task	PASS/FAIL per artifact + lineage integrity cl
6 - Sign-off	supervisor_agent	supervise_migration_task	Final Migration Completion Report

Core architectural invariant: BRONZE objects always before SILVER, SILVER always before GOLD. The supervisor enforces this via topological sort. DISCARD objects are removed from the sequence entirely - they require no code generation and no validation.

2. Procedure & Function Migration Strategy

This is the most important section of this document. Oracle procedures exist to orchestrate data movement that dbt handles automatically. The migration strategy for each procedure is determined by its `proc_pattern` — detected during discovery by inspecting the PL/SQL body.

proc_pattern	migration_strategy	dbt Artifact	Rationale
TABLE_REFRESH	DISCARD	None	TRUNCATE+INSERT, DELETE+INSERT, INSERT OVERWRITE
MERGE_UPSERT	INCREMENTAL_MODEL	models/silver/ or gold/ .sql (incremental)	MERGE INTO logic becomes a dbt incremental model with merge
SCD_TYPE_2	DBT_SNAPSHOT	snapshots/ _snapshot.sql	History columns (valid_from, valid_to, is_current) map to dbt snapshot
HELPER_FUNCTION	DBT_MACRO	macros/ .sql	Oracle scalar functions returning expressions become Jinja macros
UNIQUE_LOGIC	DBT_MODEL	models/silver/ or gold/ .sql (table or incremental)	Business logic not captured in any table or view. LOW -> table
Ambiguous / multiple	HUMAN_REVIEW	None	Added to human_review_queue with all detected patterns listed

DISCARD is a success metric, not a gap. A TABLE_REFRESH procedure that is discarded means one less artifact to build, test, and maintain forever. The equivalent dbt model for the target table handles the refresh automatically on every dbt run. Report DISCARDS prominently in all summaries as 'Replaced by dbt run cycle'.

proc_pattern Detection Rules

TABLE_REFRESH	Body primarily contains TRUNCATE+INSERT, DELETE+INSERT, INSERT OVERWRITE, or bulk INSERT with no MERGE or SCD2 logic.
MERGE_UPSERT	Body contains MERGE INTO ... USING ... ON ... WHEN MATCHED / WHEN NOT MATCHED pattern.
SCD_TYPE_2	Body maintains history columns: effective_date, end_date, is_current, valid_from, valid_to, or equivalent SCD2 conventions.
HELPER_FUNCTION	Object type = FUNCTION. Returns a scalar value or expression. Body contains no DML on target tables.
UNIQUE_LOGIC	Body contains business transformation logic not captured in any existing table or view definition.
Multiple patterns	Tag all detected patterns, note the primary. Flag to classifier as ambiguous -> HUMAN_REVIEW.

3. Medallion Architecture

Every Oracle table and view is assigned to exactly one medallion layer. The layer determines the dbt folder, materialisation strategy, and reference pattern.

BRONZE - Raw / Landing	SILVER - Transformation	GOLD - Reporting / KPI
Oracle name starts with SRC or src .	Transforms or enriches Bronze data.	Aggregates Silver data for reporting.
Declared in <code>models/bronze/sources.yml</code> .	Placed under <code>models/silver/</code> .	Placed under <code>models/gold/</code> .
Referenced via <code>source()</code> .	Always uses <code>ref()</code> — never <code>source()</code> .	Always uses <code>ref()</code> — never <code>source()</code> .
Physical BigQuery table created via DDL.	Includes Oracle views, <code>UNIQUE_LOGIC</code> procs reading from Bronze, and <code>INCREMENTAL_MODEL</code> targets.	KPIs, dashboards, fact/dim tables for consumers.
dbt model: thin <code>SELECT</code> wrapper.		

dbt Project Folder Structure

```

dbt_project/
models/
bronze/
sources.yml # SRC/src source declarations ONLY
src_customers.sql # thin source() wrapper
src_orders.sql
silver/
schema.yml
orders_enriched.sql # ref('src_orders') - Oracle view or UNIQUE_LOGIC proc
orders_upserted.sql # INCREMENTAL_MODEL from MERGE_UPSERT proc
gold/
schema.yml
revenue_summary.sql # ref('orders_enriched')
snapshots/
schema.yml
customers_snapshot.sql # DBT_SNAPSHOT from SCD_TYPE_2 proc
macros/
calc_discount.sql # DBT_MACRO from HELPER_FUNCTION
format_date.sql

```

source() vs ref() hard rule: `source()` is permitted ONLY inside `models/bronze/`. Every Silver model, Gold model, and snapshot uses `ref()` exclusively. Any violation is treated as a blocking validation failure (reason=INVALID_REF).

4. Agent Roles & Responsibilities

supervisor_agent

Migration Project Supervisor & Quality Gate Enforcer

- * Orchestrates all five specialist agents in strict pipeline order.
 - * Performs pre-flight connectivity checks (Oracle + BigQuery) before delegating.
 - * Enforces 5 quality gates — halts and escalates on any failure.
 - * Tracks DISCARD count as a success metric throughout the pipeline.
 - * Maintains and publishes the human_review_queue after each phase.
 - * Produces the Final Migration Completion Report with 8 structured sections.
 - * The ONLY agent with allow_delegation: true. All others are false.
 - * Tools (5 only): read_migration_log, write_migration_log, read_config, list_tables (1 call), count_rows_bq (1 call). max_iter: 50.
-

discovery_agent

Oracle Discovery & Procedure Pattern Analyst

- * Calls list_tables, list_views, list_procedures to enumerate all Oracle objects.
 - * Calls get_table_ddl / get_view_ddl for every table and view.
 - * Calls get_proc_body + get_dependencies for every procedure and function.
 - * Tags each procedure/function with proc_pattern: TABLE_REFRESH, MERGE_UPSERT, SCD_TYPE_2, HELPER_FUNCTION, or UNIQUE_LOGIC.
 - * Tags each table/view with medallion_layer: BRONZE (SRC/src prefix) or SILVER_CANDIDATE.
 - * Never guesses — reports absent objects as absent. max_iter: 20.
-

classifier_agent

Migration Strategy Classifier & Medallion Layer Strategist

- * Assigns migration_strategy per proc_pattern: DISCARD / INCREMENTAL_MODEL / DBT_SNAPSHOT / DBT_MACRO / DBT_MODEL.
 - * Immediately marks TABLE_REFRESH procedures as DISCARDED — no further processing.
 - * Assigns complexity tier (LOW / MEDIUM / HIGH) to all non-discarded procedural objects.
 - * Assigns final medallion_layer (BRONZE / SILVER / GOLD) to tables, views, and DBT_MODEL procedures.
 - * Produces topologically sorted migration order: Bronze first, then Silver, then Gold. DISCARDS removed.
 - * Conservative scorer — when in doubt, scores higher not lower. max_iter: 10.
-

schema_conversion_agent

Oracle-to-BigQuery DDL Converter & dbt Project Architect

- * Creates BigQuery DDL tables for Bronze (SRC/src) objects ONLY.
 - * Generates models/bronze/sources.yml with SRC/src entries only.
 - * Generates .sql stubs under models/bronze/, models/silver/, models/gold/ for tables and views.
 - * Does NOT create artifacts for procedure-derived tables — codegen_agent handles those.
 - * Bronze models use source(); Silver and Gold models use ref() exclusively.
 - * Applies Oracle -> BigQuery type mapping from schema_map.json. max_iter: 20.
-

codegen_agent

PL/SQL to dbt Artifact Engineer (Models, Snapshots & Macros)

- * Generates NO artifact for DISCARD objects — skips them entirely.
 - * INCREMENTAL_MODEL: writes incremental dbt model with merge strategy on MERGE target table.
 - * DBT_SNAPSHOT: writes dbt snapshot with timestamp or check strategy for SCD Type 2 procedures.
 - * DBT_MACRO: writes Jinja macro for scalar Oracle functions. No config() block.
 - * DBT_MODEL: writes dbt model (table or incremental) for UNIQUE_LOGIC procedures.
 - * All Silver/Gold models and snapshots use ref() — never source() or raw table strings.
 - * Appends schema.yml / snapshots/schema.yml entries for every artifact. Batches of 10. max_iter: 30.
-

validator_agent

Validation, Reconciliation & dbt Lineage Integrity Engineer

- * Skips all DISCARDED objects — no artifact was generated, no validation needed.
 - * INCREMENTAL_MODEL / DBT_MODEL: SQL dry-run + row count reconciliation + column sample (50 rows).
 - * DBT_SNAPSHOT: SQL dry-run + history record validation (SCD2 metadata columns + historical records exist).
 - * DBT_MACRO: render + dry-run + output equivalence against Oracle function.
 - * All artifact types: lineage integrity — no source() outside Bronze, no raw strings, all ref() resolve.
 - * New failure types: SNAPSHOT_HISTORY_INVALID, MACRO_OUTPUT_MISMATCH, DDL_IN_ARTIFACT, UNRESOLVED_MACRO.
 - * 95% pass rate gate enforced by supervisor before sign-off. max_iter: 20.
-

5. Complexity Scoring Rubric

Applied ONLY to non-DISCARD objects: INCREMENTAL_MODEL, DBT_SNAPSHOT, DBT_MACRO, DBT_MODEL. TABLE_REFRESH objects are discarded before scoring. When two criteria conflict, the higher score wins.

Criterion	LOW	MEDIUM	HIGH
SQL patterns	Pure SELECT/INSERT/UPDATE	Single CURSOR, FOR loops	MERGE/DELETE, EXECUTE IMMEDIATE, dynamic SQL
Procedure calls	None	Calls 1-2 LOW procs	Calls HIGH procs or DBMS_* packages
Dynamic SQL	None	None	Any EXECUTE IMMEDIATE or DBMS_SQL
Cross-schema deps	None	None	Any (owner != schema)
DB links	None	None	Any @REMOTE_DB reference
Dependency count	0 - 3 objects	3 - 8 objects	> 8 objects
Automation confidence	>= 0.90	0.75 - 0.89	< 0.75
dbt artifact	materialized='table'	materialized='incremental'	HUMAN_REVIEW queue

6. Oracle -> BigQuery Type Mapping

schema_conversion_agent applies these mappings. Always consults schema_map.json first; this table is the fallback reference.

Oracle Type	BigQuery Type	Notes / Warnings
NUMBER(p, s)	NUMERIC(p, s)	Exact numeric - preserve precision and scale.
NUMBER	FLOAT64	WARNING: precision loss possible. Flag in report.
VARCHAR2(n)	STRING	Length hint dropped. Encoding handled by BigQuery.
CHAR(n)	STRING	Oracle CHAR pads with spaces; STRING does not.
DATE	DATETIME	Oracle DATE always contains time. Maps to DATETIME not DATE.
TIMESTAMP	TIMESTAMP	Direct mapping. Timezone preserved if present.
CLOB	STRING	10 MB practical limit. Flag large CLOBs.
BLOB	BYTES	Verify downstream binary data handling.
INTEGER	INT64	Direct mapping.
BOOLEAN	BOOL	Direct mapping.
INTERVAL YEAR TO MONTH	INT64 (months)	Store as integer months; document in schema.yml.
XMLTYPE	STRING	Store as raw XML string; document for parsing.
RAW(n)	BYTES	Binary data - verify consumer compatibility.

7. PL/SQL -> BigQuery SQL Conversion Patterns

Applied by codegen_agent to all generated artifacts. Every conversion must be documented in an inline SQL comment at the point of change.

Oracle PL/SQL Pattern	BigQuery / dbt Equivalent
<code>CURSOR FOR loop</code>	CTE or subquery
<code>MERGE INTO ... USING</code>	BigQuery <code>MERGE INTO ... USING (-> incremental model)</code>
<code>NVL(a, b)</code>	<code>IFNULL(a, b)</code>
<code>NVL2(a, b, c)</code>	<code>IF(a IS NOT NULL, b, c)</code>
<code>DECODE(x, v1, r1, ...)</code>	<code>CASE WHEN x=v1 THEN r1 ... END</code>
<code>SYSDATE</code>	<code>CURRENT_DATETIME()</code>
<code>SEQUENCE.NEXTVAL</code>	<code>GENERATE_UUID() or FARM_FINGERPRINT(...)</code>
<code>ROWNUM</code>	<code>ROW_NUMBER() OVER (ORDER BY ...)</code>
<code>a b (concat)</code>	<code>CONCAT(a, b)</code>
<code>TO_DATE(str, fmt)</code>	<code>PARSE_DATETIME(fmt, str)</code>
<code>TO_CHAR(date, fmt)</code>	<code>FORMAT_DATETIME(fmt, datetime_col)</code>
<code>TRUNC(date)</code>	<code>DATE(datetime_col)</code>
<code>ADD_MONTHS(d, n)</code>	<code>DATE_ADD(d, INTERVAL n MONTH)</code>
<code>MONTHS_BETWEEN(d1, d2)</code>	<code>DATE_DIFF(d1, d2, MONTH)</code>
<code>INSTR(s, sub)</code>	<code>STRPOS(s, sub)</code>
<code>RAISE_APPLICATION_ERROR</code>	Remove; replace with dbt test or comment
<code>DBMS_OUTPUT.PUT_LINE</code>	Remove entirely
<code>IN/OUT parameters</code>	Resolved as CTEs or SELECT expressions in model body
<code>SCD2 history columns</code>	-> dbt snapshot (timestamp or check strategy)

8. dbt Artifact Templates

8.1 Incremental Model (MERGE_UPSERT)

```

{{
  config(
    materialized='incremental',
    unique_key='<key_from_MERGE_ON_clause>',
    incremental_strategy='merge',
    tags=['silver'] -- or ['gold']
  )
}}
-- Converted from Oracle procedure: PROC_NAME
-- Pattern: MERGE INTO replaced by dbt incremental materialisation
with source as (
  select ...
  from {{ ref('<upstream_bronze_or_silver_model>') }}
  {% if is_incremental() %}
  where updated_at > (select max(updated_at) from {{ this }})
  {% endif %}
)
select <explicit column list> from source

```

8.2 dbt Snapshot (SCD_TYPE_2)

```

{% snapshot customers_snapshot %}
{{
  config(
    target_schema='my_bq_dataset',
    unique_key='customer_id',
    strategy='timestamp', -- use 'check' if no updated_at column
    updated_at='updated_at', -- check_cols=['col1','col2'] for check strategy
    tags=['snapshot', 'scd2']
  )
}}
-- Converted from Oracle procedure: PROC_SCD_CUSTOMERS
-- Pattern: SCD Type 2 history tracking replaced by dbt snapshot
select <explicit column list>
from {{ ref('src_customers') }}
{% endsnapshot %}

```

8.3 dbt Macro (HELPER_FUNCTION)

```

{#
  Converted from Oracle function: CALC_DISCOUNT
  Parameters: base_price (NUMBER), discount_pct (NUMBER)
  Returns: discounted price as FLOAT64
  #}
{% macro calc_discount(base_price, discount_pct) %}
ROUND({{ base_price }} * (1 - IFNULL({{ discount_pct }}, 0) / 100), 2)
{% endmacro %}

-- Usage in a model: {{ calc_discount('unit_price', 'disc_pct') }}

```

8.4 dbt Model (UNIQUE_LOGIC)

```
{{
  config(
    materialized='table', -- LOW: table | MEDIUM: incremental
    tags=['silver']
  )
}}
-- Converted from Oracle procedure: PROC_ENRICH_ORDERS
-- Pattern: UNIQUE_LOGIC - business logic not captured in existing tables
-- Conversion notes: CURSOR FOR loop at line 12 -> CTE; NVL -> IFNULL
with orders as (
  select * from {{ ref('src_orders') }}
),
customers as (
  select * from {{ ref('src_customers') }}
),
enriched as (
  select
    o.order_id,
    o.customer_id,
    c.customer_name,
    IFNULL(o.discount_pct, 0) as discount_pct, -- NVL -> IFNULL
    CURRENT_DATETIME() as processed_at, -- SYSDATE -> CURRENT_DATETIME()
    {{ calc_discount('o.unit_price', 'o.discount_pct') }} as final_price
  from orders o
  join customers c on o.customer_id = c.customer_id
)
select * from enriched
```

9. Validation Protocol

validator_agent skips all DISCARDED objects. For every generated artifact, it runs artifact-type-specific checks plus universal lineage integrity checks. All applicable checks must pass before status=VALIDATED is set.

INCREMENTAL_MODEL & DBT_MODEL	* CHECK 1: SQL syntax dry-run via dry_run_bq. FAIL -> SYNTAX_ERROR. * CHECK 2: Row count reconciliation. $\text{diff_pct} = \frac{\text{abs}(\text{oracle-bq})}{\text{oracle}} \times 100$. FAIL -> COUNT_MISMATCH if $> 0.01\%$. * CHECK 3: Column sample 50 rows by PK. FAIL -> DATA_MISMATCH if $> 2\%$ of rows differ on any non-nullable column.
DBT_SNAPSHOT	* CHECK 1: Render snapshot SELECT and dry_run_bq. FAIL -> SYNTAX_ERROR. * CHECK 2: Verify dbt_scd_id, dbt_valid_from, dbt_valid_to, dbt_updated_at columns exist. Verify ≥ 1 history record (dbt_valid_to IS NOT NULL) for any updated source row. FAIL -> SNAPSHOT_HISTORY_INVALID.
DBT_MACRO	* CHECK 1: Render macro with sample arguments and dry_run_bq. FAIL -> SYNTAX_ERROR. * CHECK 2: execute_oracle_function with same arguments. Compare outputs. FAIL -> MACRO_OUTPUT_MISMATCH.
ALL artifact types (Lineage Integrity)	* No raw dataset.table strings anywhere in artifact SQL. FAIL -> INVALID_REF. * source() only in Bronze model files. FAIL -> INVALID_REF. * Every ref("") resolves to existing file under models/. FAIL -> INVALID_REF. * No DDL statements (CREATE, INSERT, etc.) in any artifact. FAIL -> DDL_IN_ARTIFACT. * Every {{ macro_name(...) }} in a model resolves to a file in macros/. FAIL -> UNRESOLVED_MACRO.

retry_count < 3	status=FAILED_VALIDATION. codegen_agent retries with exact failure reason as corrective prompt.
retry_count >= 3	status=HUMAN_REVIEW. Added to human_review_queue with full failure history.
PASS criteria	All applicable checks pass. confidence=1.0, status=VALIDATED.
Supervisor pass gate	Minimum 95% validation pass rate of generated (non-discarded) objects before sign-off.
DISCARD objects	Skipped entirely. No checks run. status=DISCARDED is terminal.

10. Quick Reference Card

Procedure Migration Decision Tree

TABLE_REFRESH	DISCARD. Set status=DISCARDED, discard_reason='Replaced by dbt run cycle'. No artifact. No validation.
MERGE_UPSERT	-> INCREMENTAL_MODEL. Incremental dbt model with merge strategy. unique_key from MERGE ON clause.
SCD_TYPE_2	-> DBT_SNAPSHOT. Timestamp strategy if updated_at exists, else check strategy.
HELPER_FUNCTION	-> DBT_MACRO. Jinja macro in macros/. No config() block. Called as {{ macro_name(...) }}.
UNIQUE_LOGIC	-> DBT_MODEL. LOW -> table. MEDIUM -> incremental. Layer from dependency chain.
Ambiguous	-> HUMAN_REVIEW. List all detected patterns. Classifier does not guess.

source() vs ref() Rule

Use source()	ONLY in models/bronze/ files. {{ source('dataset', 'table') }}
Use ref()	In ALL Silver, Gold, snapshot, and macro-calling model files. {{ ref('model_name') }}
Never use	Raw dataset.table strings in any dbt artifact.
Violation code	INVALID_REF — blocking validation failure.

Failure Type Reference

SYNTAX_ERROR	BigQuery dry-run rejected the SQL.
COUNT_MISMATCH	Row count difference > 0.01% between Oracle and BigQuery.
DATA_MISMATCH	More than 2% of sampled rows differ on a non-nullable column.
SNAPSHOT_HISTORY_INVALID	SCD2 metadata columns missing or no history records found.
MACRO_OUTPUT_MISMATCH	Macro output differs from Oracle function output on same inputs.
INVALID_REF	source() in Silver/Gold, unresolved ref(), or raw table string.
DDL_IN_ARTIFACT	CREATE/INSERT/DDI found in a model, snapshot, or macro file.
UNRESOLVED_MACRO	{{ macro_name() }} in a model has no matching file in macros/.
ORACLE_UNREACHABLE	Pre-flight: Oracle connectivity check failed.
BIGQUERY_UNREACHABLE	Pre-flight: BigQuery connectivity check failed.

Agent Configuration Summary

Agent	allow_delegation	max_iter	Key Constraint
supervisor_agent	TRUE	50	Only agent that can delegate. 5 tools only. Enforces all gates.
discovery_agent	false	20	Tags every proc with proc_pattern. Never guesses.
classifier_agent	false	10	DISCARD before scoring. Conservative. Topo sort enforced.
schema_conversion_agent	false	20	DDL for Bronze only. sources.yml: SRC/src only.
codegen_agent	false	30	No artifact for DISCARDS. Correct artifact per strategy. Batches
validator_agent	false	20	Skip DISCARDS. Type-specific checks. 95% gate. Lineage enfo

This document is generated from the authoritative agents.yaml, tasks.yaml, and supervisor_agent.yaml files. In case of any discrepancy, the YAML files take precedence. Regenerate this document whenever the YAML files are modified.